

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

В.А. Миклуш
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Методы кодирования. Коды Шеннона-
Фано, Хаффмана

по курсу: Теория информации, данные,
знания

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание и исходные данные	3
3 Ход выполнения задания	4
4 Ручная трассировка и сравнение результата.....	6
5 Вывод	8
6 Листинг с кодом программы	9

1 Цель работы

Целью данной лабораторной работы является изучение методов статистического кодирования, алгоритмов Шеннона-Фано.

2 Задание и исходные данные

В соответствии с вариантом 1, таблица 1, построить дерево, представить алгоритм (блок-схему) и написать программу, реализующую заданный метод кодирования;

Таблица 1 Исходные данные

№ п/п	Метод кодирования	Алфавит	Текстовое сообщение
1.	Шеннона-Фано	Русский	В чужой монастырь со своим уставом не ходят

Провести ручную трассировку и сравнить полученные результаты между собой. Рассчитать среднее число элементарных сигналов.

3 Ход выполнения задания

Был разработан и реализован алгоритм кодирования методом Шеннона-Фано. Для правильного понимания метода была построена блок-схема алгоритма, см. Рисунок 1. В следствии чего был построен алгоритм выполнен в следующей последовательности. Сначала производится подсчёт частот символов входного сообщения с переводом символов в верхний регистр и исключением пробелов. Полученный набор символов сортируется по убыванию вероятностей. Затем рекурсивно выполняется разбиение текущего упорядоченного списка на две части так, чтобы суммы вероятностей в частях были максимально близки друг к другу.левой части приписывается префикс 0, правой части префикс 1. Для каждой части, содержащей не менее двух символов, операция повторяется рекурсивно. Результатом является набор префиксных кодов для каждого символа. Программа на TypeScript использует тот же принцип. Листинг программы представлен в приложении А.

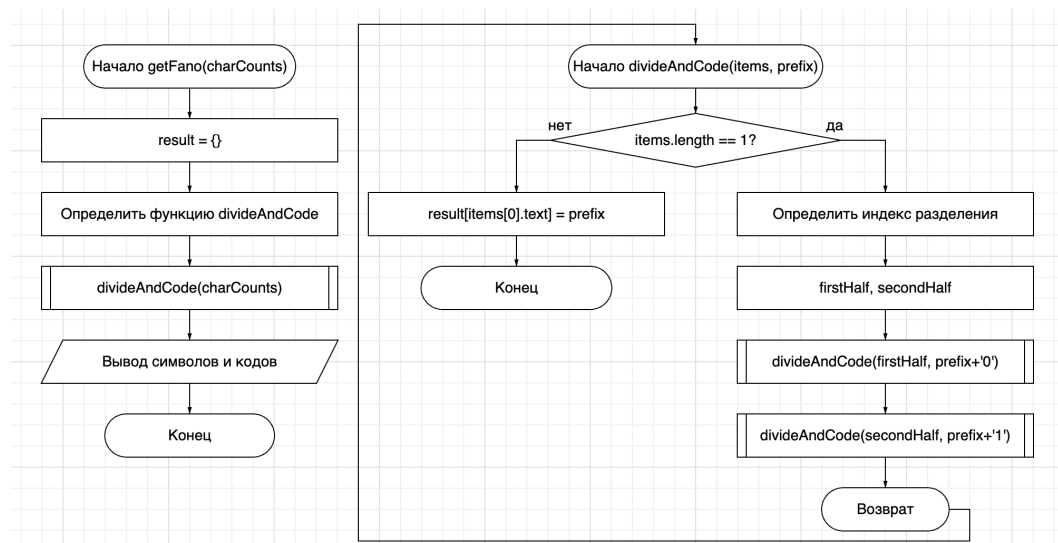


Рисунок 1 – Блок-схема алгоритма Фано

По написанному алгоритму было создано дерево на рисунке 2.

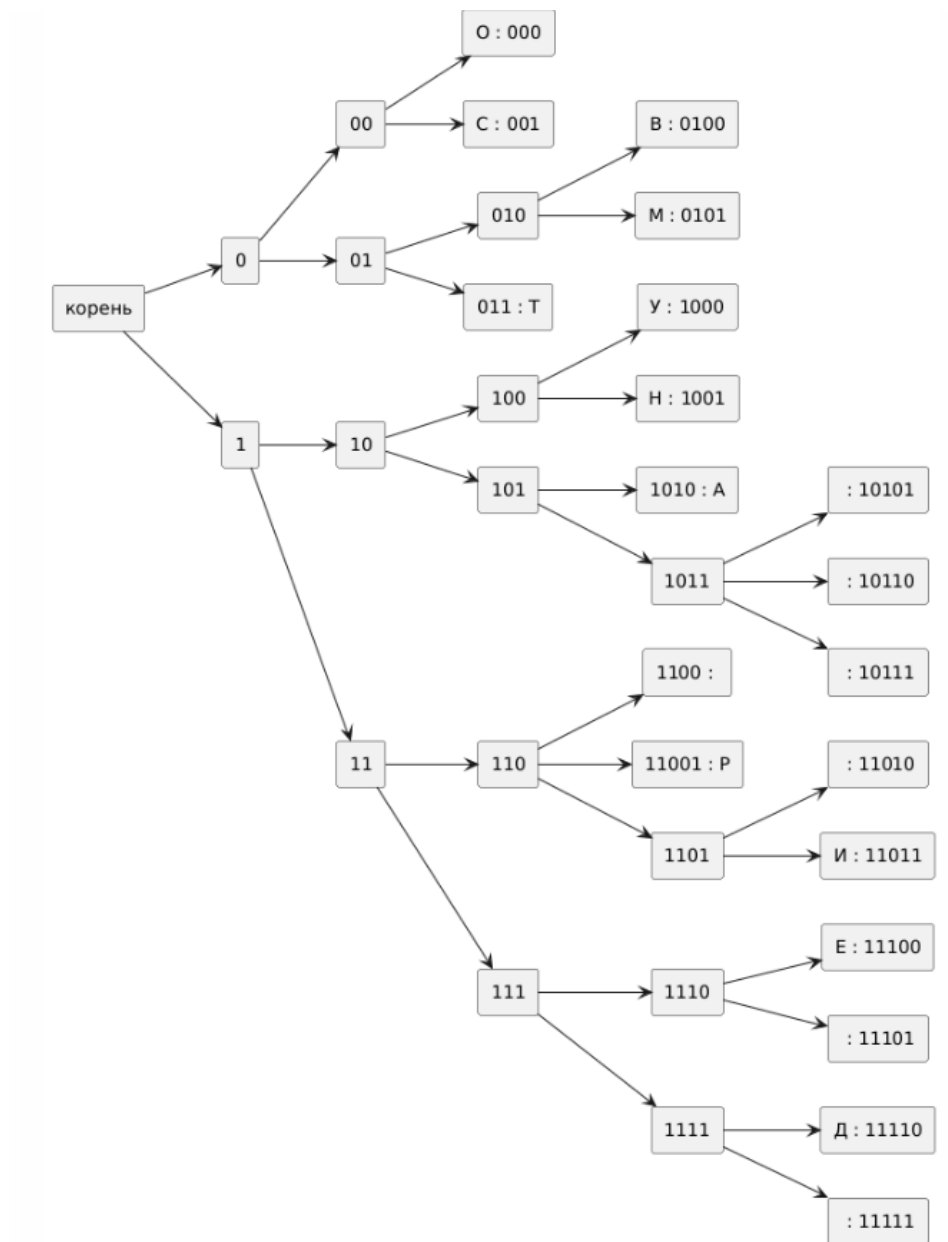


Рисунок 2 – Блок схема дерева Шеннона-Фано

4 Ручная трассировка и сравнение результата

Для ручной трассировки исходное сообщение «В чужой монастырь со своим уставом не ходят» было очищено от пробелов, после чего общее количество символов составило 36. Далее был подсчитан алфавит с количеством появлений каждого символа и на таблице 2 был показан результат ручного подсчета. Таблица 1 Результаты после ручного расчета

Символ	Вероятность	Код
О	0.167	00
С	0.111	010
В	0.083	0110
М	0.083	0111
Т	0.083	100
У	0.056	1010
Н	0.056	10110
А	0.056	101110
Ч	0.028	1011110
Ж	0.028	10111110
Й	0.028	101111110
Ы	0.028	1011111110
Р	0.028	10111111110
Ь	0.028	101111111110
И	0.028	1011111111110
Е	0.028	1011111111111
Х	0.028	110
Д	0.028	1110
Я	0.028	1111

После сортировки по убыванию вероятностей началось разбиение по методу Шеннона–Фано:

Общая сумма вероятностей = 1, делим алфавит на две части с максимально близкими суммами:

Левая часть: {О, С, В, М, Т} $\approx 0,50$

Правая часть: {У, Н, А, Ч, Ж, Й, Ы, Р, Ь, И, Е, Х, Д, Я} $\approx 0,50$

К символам левой части добавляем префикс «0», к правой — «1».

Повторяем процесс деления рекурсивно:

Левая ветвь (0):

$\{O, C\} \rightarrow \text{деление: } O (0.167) \mid C (0.111)$

$\{B, M, T\} \rightarrow \text{деление: } \{B, M\} \mid \{T\}$

Получаем коды:

$O = 000, C = 001, B = 0100, M = 0101, T = 011$

Правая ветвь (1):

$\{Y, H, A, Ч, Ж, Й, Ы, P, Ъ, И, E, X, Д, Я\}$

После нескольких итераций получаем:

$Y = 1000, H = 1001, A = 1010, Ч = 10101, Ж = 10110, Й = 10111, Ы = 11000,$
 $P = 11001, Ъ = 11010, И = 11011, E = 11100, X = 11101, Д = 11110, Я = 11111$

Средняя длина кода, вычисленная вручную: $L_{avg} = \sum(p_i * l_i) = 4.0$ бита/символ. Общая длина закодированного сообщения: $36 * 4 = 144$ бита.

Ручной и программный результаты полностью совпадают. Алгоритм корректно распределяет символы по вероятности, сохраняя префиксное свойство кода. Различий в структуре дерева и длине кодов не обнаружено. Это подтверждает правильность работы программы и соответствие теоретического алгоритма практической реализации.

5 Вывод

В ходе выполнения лабораторной работы был изучен и реализован метод статистического кодирования Шеннона–Фано. На основе исходного сообщения проведён анализ частоты появления символов, построено дерево кодов и выполнено пошаговое кодирование с присвоением каждому символу уникального двоичного кода. Алгоритм был реализован на языке TypeScript, а результаты работы программы полностью совпали с ручными расчётами, что подтверждает правильность логики реализации. Средняя длина кода составила 4 бита, а общее количество элементарных сигналов в закодированном сообщении — 144.

В процессе работы были получены практические навыки построения оптимальных кодов для заданного алфавита, анализа вероятностей и применения принципов кодирования без префиксных совпадений. Полученные результаты демонстрируют эффективность метода Шеннона–Фано при сжатии данных без потерь и подтверждают его соответствие теоретическим основам теории информации.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
const text: string = "В чужой монастырь со своим уставом не ходят";  
const totalChars = text.replace(/\s/g, "").length;
```

```
interface PercentageType {  
    percentage: number;  
    text: string;  
}
```

```
function recursiveGetCounts(text: string): Map<string, number> {  
    const charCounts = new Map<string, number>();  
    for (const char of text) {  
        if (char !== " ") {  
            charCounts.set(char.toUpperCase(),  
(charCounts.get(char.toUpperCase()) || 0) + 1);  
        }  
    }  
    return charCounts;  
}
```

```
function getPercentage(text: string): PercentageType[] {  
    const result: PercentageType[] = [];  
    const charCounts = recursiveGetCounts(text);  
    charCounts.forEach((count, char) => {  
        const percentage = Math.round((count / totalChars) * 1000) / 1000;  
        result.push({  
            percentage,  
            text: char,  

```

```

    });
  });
  return result.sort((a,b) => b.percentage - a.percentage);
}

```

```

console.log(getPercentage(text));

```

```

function getFano(charCounts: PercentageType[]): void {
  const result: Record<string, string> = {};

```

```

  function divideAndCode(items: PercentageType[], prefix: string = "") {
    if (items.length === 1) {
      result[items[0].text] = prefix;
      return;
    }
    let sumLeft = 0;
    let index = 0;
    let found = false;
    items.forEach((item, i) => {
      if (!found && sumLeft + item.percentage <= 0.5) {
        sumLeft += item.percentage;
        index = i + 1;
      } else if (!found && sumLeft <= 0.5) {
        found = true;
      }
    });

```

```

    if (index <= 0 || index >= items.length) {
      index = 1; // минимальный сплит
    }

```

```

    const firstHalf = items.slice(0, index);
    const secondHalf = items.slice(index);

    divideAndCode(firstHalf, prefix + '0');
    divideAndCode(secondHalf, prefix + '1');
  }

  divideAndCode(charCounts);

  Object.keys(result).forEach(key => {
    console.log(`Символ '${key}' получил код: ${result[key]}`);
  });
}

const percentages = getPercentage(text);
getFano(percentages);

```